Machine Saathi: Graph-Based Machine Fault Impact & Dependency Analyzer

Student Name: Avani Jain | PRN: 25070126044
Department of Artificial Intelligence and Machine Learning | Symbiosis Institute of Technology, Pune

Problem Statement

Industrial machines contain interconnected components, where a fault in one component may affect other dependent components. Understanding these relationships manually can be difficult, especially when dependencies form complex networks.
Machine Saathi models machine components and their dependencies using a directed graph to help users explore potential fault impact and dependency paths.

Objectives

- Represent machine components and their dependencies using a directed graph.
- Implement an adjacency list for storing component relationships.
- Apply BFS to identify downstream components by graph-distance level.
- Apply DFS to explore possible dependency paths.
- Support component searches, fault reporting, dependency updates and machine-status management.

Solution

Machine Saathi provides a graph-based approach to understanding machine-component dependencies and analysing the potential impact of component faults.

- Dependency Modelling: Represents machine components as vertices and their dependencies as directed edges.
- Fault Impact Analysis: Uses BFS to identify downstream components that may be affected, organized by graph-distance levels.
- Dependency Path Tracing: Uses DFS to explore possible paths from a faulty component through the dependency graph.
- Interactive Management: Enables users to configure machine components, update dependencies, search components, record faults and reset machine status.

This approach provides an interpretable view of potential fault impact based on the dependencies configured by the user.

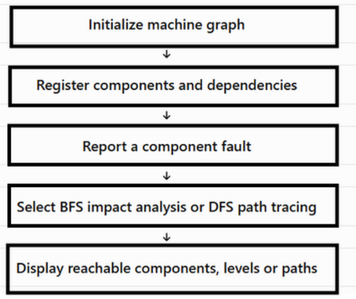
Data Structures, Algorithms & Tools Used

- Directed Graph: Represents each machine component as a vertex and each dependency as a directed edge.
- Adjacency List: Stores the outgoing dependencies of each component using linked-list nodes.
- Breadth-First Search (BFS): Uses a queue and visited/level information to explore downstream dependencies level by level.
- Depth-First Search (DFS): Uses recursion and backtracking to explore possible dependency paths.
- Additional structures: Arrays, component records/structures, and dynamically allocated graph nodes support component management and traversal.
- Programming language: C
- Compiler: GCC
- Core concepts: Graphs, adjacency lists, linked lists, queues, arrays, recursion, dynamic memory allocation.

Methodology

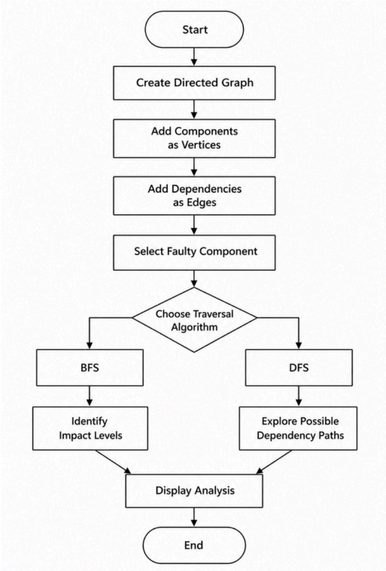
Machine Saathi models machine components as vertices in a directed graph, with edges representing dependency relationships. The graph is stored using an adjacency-list representation, allowing users to configure components and update dependencies.
When a component fault is registered, BFS uses a queue and visited information to identify reachable downstream components and organize them by graph distance. DFS uses recursion and backtracking to explore possible dependency paths. The menu-driven program displays the analysis results and supports component searches, dependency lookup, machine-status management, and configuration updates.

The following steps illustrate how Machine Saathi models component dependencies and performs graph-based fault-impact analysis:



Graph Representation

Machine Saathi represents machine components as vertices and their dependency relationships as directed edges in a graph stored using an adjacency list. After a user selects a faulty component, BFS identifies reachable downstream components level by level, while DFS explores possible dependency paths. The resulting analysis helps users understand potential fault impact based on the configured graph.
the following is the workflow:



Results & Complexity Analysis

Results
Machine Saathi uses a directed graph to represent machine-component dependencies. BFS identifies reachable downstream components according to their shortest graph distance, while DFS explores possible dependency paths. The adjacency-list representation supports component relationships, and users can modify machine configurations and dependencies through the menu-driven interface.

Complexity Analysis

Operation	Time / Space Complexity
BFS traversal	(O(V+E)) time
Graph storage (adjacency list)	(O(V+E)) space
BFS auxiliary space	(O(V))
DFS path enumeration	Potentially exponential time

Here, (V) represents the number of vertices (components) and (E) represents the number of edges (dependencies). DFS path enumeration can take exponential time in the worst case because multiple possible paths may need to be explored.

Conclusion & Future Scope

Conclusion
Machine Saathi demonstrates the practical application of directed graphs, adjacency lists, BFS, and DFS to machine-component dependency analysis. BFS identifies reachable downstream components by graph-distance levels, while DFS explores possible dependency paths. This provides an interpretable way to examine potential fault impact using user-defined machine configurations.

Future Scope
The system can be extended by integrating real-time sensor data, monitoring machine conditions, and investigating automated fault detection. These enhancements could help develop a more comprehensive predictive-maintenance system beyond the current graph-based implementation.

References

[1] Y. Qin, H. Wang, and Y. Li, “Toward cognitive predictive maintenance: A survey of graph-based approaches,” Computers in Industry, vol. 144, 2023.
[2] F. Yang, “Signed directed graph-based hierarchical modelling and fault propagation analysis for large-scale systems,” IET Control Theory & Applications, vol. 7, no. 2, 2013, doi: 10.1049/iet-cta.2010.0660.
[3] T. H. Cormen, C. E. Leiserson, R. L. Rivest, and C. Stein, Introduction to Algorithms, 4th ed. MIT Press, 2022.

Implementation Results

```
Enter component name: Motor
BFS FAILURE IMPACT ANALYSIS
------------------------------------------------
Failed Component: Motor
Criticality: 5

Affected Components:

Level 1:
  Shaft - AT RISK - Criticality 4
  Control Unit - AT RISK - Criticality 5

Level 2:
  Compressor Unit - AT RISK - Criticality 5

Level 3:
  Pressure Sensor - AT RISK - Criticality 2
```

```
Enter component name: Shaft

------------------------------------------------
DFS FAILURE PROPAGATION PATHS
------------------------------------------------
Starting Component: Shaft

Path 1: Shaft -> Compressor Unit -> Pressure Sensor
------------------------------------------------
```